ISO/IEC JTC1 SC22 WG21 P1131R0
Jens Maurer <Jens.Maurer@gmx.net>
Target audience: CWG
2018-06-23
☐ Hide inserted text ☐ Hide deleted text

P1131R0: Core Issue 2292: simple-template-id is ambiguous between class-name and type-name

This paper presents the wording changes to resolve core issue 2292. The issue description says:

> The grammar term *simple-template-id* is used in the definition of both *class-name* (12 [class] paragraph 1) and *type-name* (10.1.7.2 [dcl.type.simple] paragraph 1). The latter case is intended to apply to alias template specializations. It would be helpful to have separate grammar terms for these uses.

As directed by CWG, this paper restricts the use of the grammar term *class-name* to situations where the identifier introduced in a *class-head* is named. This mostly applies to declarations of classes, constructors, and destructors. Furthermore, the grammar for pseudo-destructor calls is integrated into the grammar for regular destructor calls.

Proposed wording

Change 6.4.5 [basic.lookup.classref] paragraph 2:

> If the *id-expression* in a class member access (8.5.1.5) is an *unqualified-id*, and the type of the object expression is of a class type C, the *unqualified-id* is looked up in the scope of class C. ~~For a pseudo-destructor call (8.5.1.4), the unqualified-id is looked up in the context of the complete postfix-expression.~~
>
> If *the unqualified-id* is ~ *type-name*, the *type-name* is looked up in the context of the entire *postfix-expression*. If the type T of the object expression is of a class type C, the *type-name* is also looked up in the scope of class C. At least one of the lookups shall find a name that refers to cv T. [Example: ...]

Change in 8.4.4.1 [expr.prim.id.qual] paragraph 1:

> ```
> unqualified-id:
>     identifier
>     operator-function-id
>     conversion-function-id
>     literal-operator-id
>     ~ class-name  type-name
>     ~ decltype-specifier
>     template-id
> ```
>
> ... [Note: For *operator-function-id*s, see 16.5; for *conversion-function-id*s, see 15.3.2; for *literal-operator-id*s, see 16.5.8; for *template-id*s, see 17.2. A ~~*class-name*~~ ***type-name*** or *decltype-specifier* prefixed by ~ denotes ~~a destructor~~ **destruction of the type so named**; see ~~15.4 [class.dtor]~~ **8.4.4.3 [expr.prim.id.dtor]**. Within the definition of a non-static member function, an identifier that names a non-static member is transformed to a class member access expression (12.2.2). -- end note]

Change in 8.4.4.2 [expr.prim.id.qual] paragraph 2:

... Where ~~*class-name* **::~** *class-name*~~ ***type-name* ::~ *type-name*** is used, the two ~~*class-names*~~ ***type-names*** shall refer to the same ~~class~~ **type (ignoring cv-qualifications)**; this notation ~~names the destructor (15.4 [class.dtor])~~ **denotes destruction (8.4.4.3 [expr.prim.id.dtor]) of that type**. ~~The form ~ *decltype-specifier* also denotes the destructor, but it shall not be used as the~~ **The** *unqualified-id* in a *qualified-id* **shall not be of the form ~ *decltype-specifier***. ~~[Note: A *typedef-name* that names a class is a *class-name* (12.1). -- end note]~~

Add a new section 8.4.4.3 [expr.prim.id.dtor]:

8.4.4.3 Destruction [expr.prim.id.dtor]

An *id-expression* that denotes destruction of a type T names the destructor of T if T is a class type (15.4 [class.dtor]), otherwise the *id-expression* is said to name a *pseudo-destructor*.

If the *id-expression* names a pseudo-destructor, T shall be a scalar type and the *id-expression* shall appear as the right operand of a class member access (8.5.1.5 [expr.ref]) that forms the *postfix-expression* of a function call (8.5.1.2 [expr.call]). [Note: Such a call has no effect. -- end note]

[Example:

```
struct C { };
void f()
{
  C * pc = new C;
  using C2 = C;
  pc->C::~C2();     // ok, destroys *pc
  C().C::~C();      // undefined behavior: temporary of type C destroyed twice
  using T = int;
  0 .T::~T();       // ok, no effect
  0.T::~T();        // error: 0. is a floating-point literal (5.13.4 [lex.fcon])
}
```

-- end example]

Change in 8.5.1 [expr.post] paragraph 1:

```
postfix-expression:
        primary-expression
        postfix-expression [ expr-or-braced-init-list ]
        postfix-expression ( expression-list_opt )
        simple-type-specifier ( expression-list_opt )
        typename-specifier ( expression-list_opt )
        simple-type-specifier braced-init-list
        typename-specifier braced-init-list
        postfix-expression . template_opt id-expression
        postfix-expression -> template_opt id-expression
        postfix-expression . pseudo-destructor-name
        postfix-expression -> pseudo-destructor-name
        postfix-expression ++
        postfix-expression --
        dynamic_cast < type-id > ( expression )
        static_cast < type-id > ( expression )
        reinterpret_cast < type-id > ( expression )
        const_cast < type-id > ( expression )
        typeid ( expression )
        typeid ( type-id )
expression-list:
        initializer-list
pseudo-destructor-name:
```

~~*nested-name-specifier*$_{opt}$ *type-name* :: ~ *type-name*~~
~~*nested-name-specifier template simple-template-id* :: ~ *type-name*~~
~~~ *type-name*~~
~~~ *decltype-specifier*~~

Change in 8.5.1.2 [expr.call] paragraph 3:

If the *postfix-expression* ~~designates~~ **names** a destructor ~~(15.4)~~ **or pseudo-destructor (8.4.4.3 [expr.prim.id.dtor])**, the type of the function call expression is void; otherwise, the type of the function call expression is the return type of the statically chosen function (i.e., ignoring the virtual keyword), even if the type of the function actually called is different. This return type shall be an object type, a reference type or cv void. **If the *postfix-expression* names a pseudo-destructor, the function call has no effect.**

Remove all of 8.5.1.4 [expr.pseudo]:

~~The use of a pseudo-destructor-name after a dot . or arrow -> operator represents the destructor for the non-class type denoted by type-name or decltype-specifier. The result shall only be used as the operand for the function call operator (), and the result of such a call has type void. The only effect is the evaluation of the postfix-expression before the dot or arrow.~~

~~The left-hand side of the dot operator shall be of scalar type. The left-hand side of the arrow operator shall be of pointer to scalar type. This scalar type is the object type. The cv-unqualified versions of the object type and of the type designated by the pseudo-destructor-name shall be the same type. Furthermore, the two type-names in a pseudo-destructor-name of the form~~

~~*nested-name-specifier*$_{opt}$ *type-name* :: ~ *type-name*~~

~~shall designate the same scalar type (ignoring cv-qualification).~~

Change in 8.5.1.5 [expr.ref] paragraphs 1-3:

A postfix expression followed by a dot . or an arrow ->, optionally followed by the keyword template (17.2), and then followed by an id-expression, is a postfix expression. The postfix expression before the dot or arrow is evaluated; [Footnote: ...] the result of that evaluation, together with the id-expression, determines the result of the entire postfix expression.

For the first option (dot) the first expression shall be a glvalue ~~having class type~~. For the second option (arrow) the first expression shall be a prvalue having pointer ~~to class~~ type. ~~In both cases, the class type shall be complete unless the class member access appears in the definition of that class. [Note: If the class is incomplete, lookup in the complete class type is required to refer to the same declaration (6.3.7). -- end note]~~ The expression E1->E2 is converted to the equivalent form (*(E1)).E2; the remainder of 8.5.1.5 will address only the first option (dot). [Footnote: ...]

Abbreviating *postfix-expression.id-expression* as E1.E2, E1 is called the *object expression*. If the object expression is of scalar type, E2 shall denote destruction of that same type (ignoring cv-qualifications) and E1.E2 is an lvalue of type "function of () returning void". [Note: This value can only be used for a notional function call (8.4.4.3 [expr.prim.id.dtor]). -- end note]

Otherwise, the object expression shall be of class type. The class type shall be complete unless the class member access appears in the definition of that class. [Note: If the class is incomplete, lookup in the complete class type is required to refer to the same declaration (6.3.7). -- end note] The ~~In either case, the~~ *id-expression* shall name a member of the class or of one of its base classes. [Note: Because the name of a class is inserted in its class scope (Clause 12), the name of a class is also considered a nested member of that class. -- end note] [Note: 6.4.5 describes how names are looked up after the . and -> operators. -- end note]

~~Abbreviating postfix-expression.id-expression as E1.E2, E1 is called the *object expression*~~. If E2 is a bit-field, E1.E2 is a bit-field. The type and value category of E1.E2 are determined as follows. In the remainder of 8.5.1.5, cq represents either const or the absence of const and vq represents either volatile or the absence of volatile. cv represents an arbitrary set of cv-qualifiers, as defined in 6.7.3.

Change in 8.5.2.1 [expr.unary.op] paragraph 10:

The operand of ~ shall have integral or unscoped enumeration type; the result is the ones' complement of its operand. Integral promotions are performed. The type of the result is the type of the promoted operand. There is an ambiguity in the grammar when ~ is followed by a ~~*class-name*~~ ***type-name*** or *decltype-specifier*. The ambiguity is resolved by treating ~ as the unary complement operator rather than as the start of an *unqualified-id* naming a destructor. [Note: Because the grammar does not permit an operator to follow the ., ->, or :: tokens, a ~ followed by a ~~*class-name*~~ ***type-name*** or *decltype-specifier* in a member access expression or *qualified-id* is unambiguously parsed as a destructor name. -- end note]

Change in 10.1.3 [dcl.typedef] paragraph 1:

```
typedef-name:
      identifier
      simple-template-id
```

A name declared with the `typedef` specifier becomes a *typedef-name*. ~~Within the scope of its declaration, a *typedef-name* is syntactically equivalent to a keyword and~~ **A *typedef-name*** names the type associated with the ~~identifier~~ ***identifier*** **(Clause 11 [dcl.decl]) or *simple-template-id* (Clause 17 [temp])** ~~in the way described in Clause 11 [dcl.decl]. A~~ **; a** *typedef-name* is thus a synonym for another type. A *typedef-name* does not introduce a new type the way a class declaration (12.1 [class.name]) or enum declaration **(10.2 [dcl.enum])** does. [Example: ...]

Change in 10.1.3 [dcl.typedef] paragraph 8:

[Note: ~~A *typedef-name* that names a class type, or a cv-qualified version thereof, is also a *class-name* (12.1 [class.name]).~~ If a *typedef-name* is used to identify the subject of an *elaborated-type-specifier* (10.1.7.3), a class definition (Clause 12), a constructor declaration (15.1), or a destructor declaration (15.4), the program is ill-formed. -- end note] [Example: ...]

Change in 10.1.7.2 [dcl.type.simple] paragraph 1:

```
type-name:
      class-name
      enum-name
      typedef-name
      simple-template-id
```

Change in 10.1.7.3 [dcl.type.elab] paragraph 2:

[Note: 6.4.4 [basic.lookup.elab] describes how name lookup proceeds for the *identifier* in an *elaborated-type-specifier*. **-- end note]** If the *identifier* **or *simple-template-id*** resolves to a *class-name* or *enum-name*, the *elaborated-type-specifier* introduces it into the declaration the same way a *simple-type-specifier* introduces its *type-name* **(10.1.7.2 [dcl.type.simple]).** If the *identifier* **or *simple-template-id*** resolves to a *typedef-name* ~~or the *simple-template-id* resolves to an alias template specialization~~ **(10.1.3 [dcl.typedef], 17.2 [temp.names])**, the *elaborated-type-specifier* is ill-formed. [Note: This implies that, within a class template with a template *type-parameter* T, the declaration `friend class T;` is ill-formed. However, the similar declaration `friend T;` is allowed (14.3). -- end note]

Change in 12 [class] paragraph 1:

```
class-name:
      identifier
      simple-template-id
```

Remove 12.1 [class.name] paragraph 5:

> ~~A *typedef-name* (10.1.3 [dcl.typedef]) that names a class type, or a cv-qualified version thereof, is also a *class-name*. If a *typedef-name* that names a cv-qualified class type is used where a *class-name* is required, the cv-qualifiers are ignored. A *typedef-name* shall not be used as the identifier in a *class-head*.~~

Change in 13 [class.derived] paragraph 1:

```
class-or-decltype:
    nested-name-specifier_opt class-name  type-name
    nested-name-specifier template simple-template-id
    decltype-specifier
```

Change in 13 [class.derived] paragraph 2:

> ~~If the name found is not a *class-name*, the program is ill-formed.~~

Change in 15.1 [class.ctor] paragraph 1:

> ~~The *class-name* shall not be a *typedef-name*.~~ In a constructor declaration, each *decl-specifier* in the optional *decl-specifier-seq* shall be friend, inline, explicit, or constexpr. [Example: ...]

Change in 15.4 [class.dtor] paragraph 1:

> ~~The *class-name* shall not be a *typedef-name*.~~ A destructor shall take no arguments (11.3.5). Each *decl-specifier* of the *decl-specifier-seq* of a destructor declaration (if any) shall be friend, inline, or virtual.

Merge and change in 17.2 [temp.names] paragraphs 6 and 7:

> A *simple-template-id* that names a class template specialization ~~is a *class-name* (Clause 12 [class]).~~ **or that is an alias template specialization, or a *simple-template-id* whose *template-name* is a template *template-parameter*, is a *typedef-name*.** ~~[Note: This rule disambiguates parsing of *simple-template-ids* between *class-name* and *typedef-name*. -- end note]~~
>
> ~~A *template-id* that names an alias template specialization is a *type-name*.~~

Change in A.1 [gram.key] paragraph 1:

```
typedef-name:
    identifier
    simple-template-id

namespace-name:
    identifier
    namespace-alias

namespace-alias:
    identifier

class-name:
    identifier
    simple-template-id
```

enum-name:
    identifier

template-name:
    identifier

~~Note that a *typedef-name* naming a class is also a *class-name* (12.1).~~